

TD Intelligence Artificielle n° 2 (Correction)

Algorithmes de jeux

Exercice 1 Jeu de Nim

On s'intéresse dans cet exercice à la modélisation d'une des nombreuses variantes du jeu de Nim, pouvant être décrite ainsi : deux joueurs enlèvent tour à tour 1, 2 ou 3 allumettes d'un tas qui en contient initialement m ; le joueur qui ramasse la dernière allumette est le perdant.

La figure 1 présente l'arbre de jeu entier, dans le cas $m = 4$ (les chiffres indiquent le nombre d'allumettes restantes). Notez que la simplicité des règles permet ici d'adopter une écriture compacte de l'arbre (qui ne se généralise pas pour d'autres types de jeux).

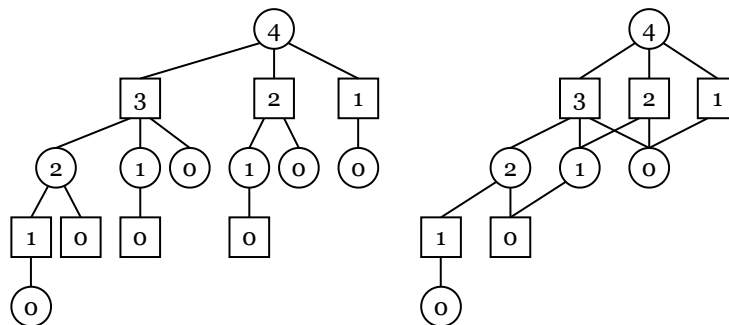
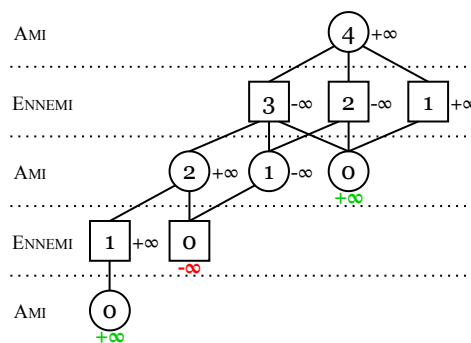


FIGURE 1 – Arbre de jeu entier pour $m = 4$ (à droite, écriture compact).

1. Sur l'arbre de la figure 1, marquez les fins de partie (c.-à-d. les feuilles) avec $+\infty$ ou $-\infty$, selon que le premier joueur (celui qui commence) gagne ou perd, respectivement. Puis, faites remonter les valeurs selon le principe de l'algorithme MinMax. Que constatez-vous ?

Correction :



On observe une stratégie gagnante pour le joueur qui commence ($+\infty$ à la racine) : s'il joue bien, sa victoire est assurée.

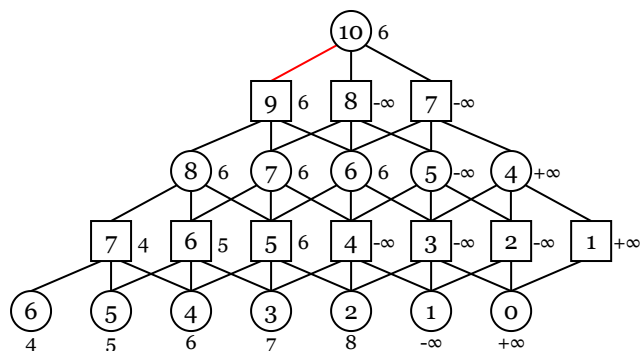
Dans les questions qui suivent, vous allez simuler l'affrontement entre deux IA :

- celle jouant en premier, que l'on nommera *Alice*, réfléchit 4 coups à l'avance (elle applique l'algorithme MinMax à profondeur 4),
- celle jouant en second, que l'on nommera *Bob*, réfléchit seulement 2 coups à l'avance (elle applique l'algorithme MinMax à profondeur 2).

On se placera dans le cas $m = 10$, et l'on supposera de plus que, concernant la fonction d'évaluation, Alice utilise $f_A = f(\text{ALICE}, \cdot)$, et Bob utilise $f_B = f(\text{BOB}, \cdot)$ (f est décrite en annexe).

2. Combien d'allumettes sont retirées par Alice lors du premier coup ? Dessinez l'arbre exploré de façon compacte.

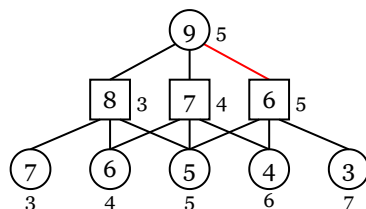
Correction : MinMax à profondeur 4 :



Au premier coup, Alice retire une allumette.

3. Combien d'allumettes sont retirées par Bob lors du deuxième coup ? Dessinez l'arbre exploré de façon compacte.

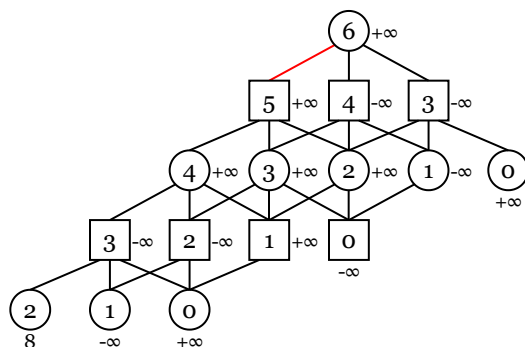
Correction : MinMax à profondeur 2 :



Au deuxième coup, Bob retire trois allumettes.

4. Combien d'allumettes sont retirées par Alice lors du troisième coup ? Dessinez l'arbre exploré de façon compacte.

Correction : MinMax à profondeur 4 :



Au troisième coup, Alice retire une allumette.

5. Quel commentaire peut-on faire à ce niveau de la partie ?

Correction : À ce niveau, Alice a une stratégie gagnante (la valeur remontée à la racine est $+\infty$). Elle a donc déjà virtuellement gagné la partie.

Annexe

Algorithme 1 Fonction d'évaluation pour Alice ou Bob

Entrée :

- *joueur* : un des deux joueurs (ALICE ou BOB)
- *n* : un nœud de l'arbre de jeu, pour lequel on dispose des méthodes *tour()*, qui renvoie le joueur dont c'est le tour (ALICE ou BOB), et *nbAllumettes()*, qui donne le nombre d'allumettes restantes

Sortie : valeur dans $\mathbb{R}$ correspondant à la qualité de *n* du point de vue de *joueur*

```
fonction f(joueur, n)  
  si n.nbAllumettes() = 0 alors  
    /* qui commence son tour avec 0 allumette a gagné */  
    si n.tour() = joueur alors  
      retourner  $+\infty$   
    sinon  
      retourner  $-\infty$   
    fin si  
  sinon si n.nbAllumettes() = 1 alors  
    /* qui commence son tour avec 1 allumette va perdre */  
    si n.tour() = joueur alors  
      retourner  $-\infty$   
    sinon  
      retourner  $+\infty$   
    fin si  
  sinon  
    retourner m - n.nbAllumettes()  
  fin si  
fin fonction
```

Exercice 2 Dominos horizontaux et verticaux

On considère le jeu suivant : sur une grille 4×4 , deux joueurs posent à tour de rôle un domino, l'un horizontalement (on parlera du « joueur horizontal ») et l'autre verticalement (on parlera du « joueur vertical »). Le dernier à poser un domino gagne la partie.

La figure 2 (à compléter) présente l'arbre de jeu à profondeur 2, dans le cas où le joueur horizontal est le premier à poser ; remarquez que pour simplifier, on a regroupé les nœuds équivalents par symétrie. Est également fournie f_H la fonction d'évaluation pour le joueur horizontal (algorithme 2). Un exemple de calcul de f_H est donné pour une des feuilles dans la figure 2.

1. Complétez les grilles non remplies de la figure 2, en tenant compte des équivalences par symétrie. Calculez f_H pour toutes les feuilles de l'arbre, puis appliquez l'algorithme MinMax. Quel coup devrait jouer le joueur horizontal, d'après cette exploration ?

Correction :

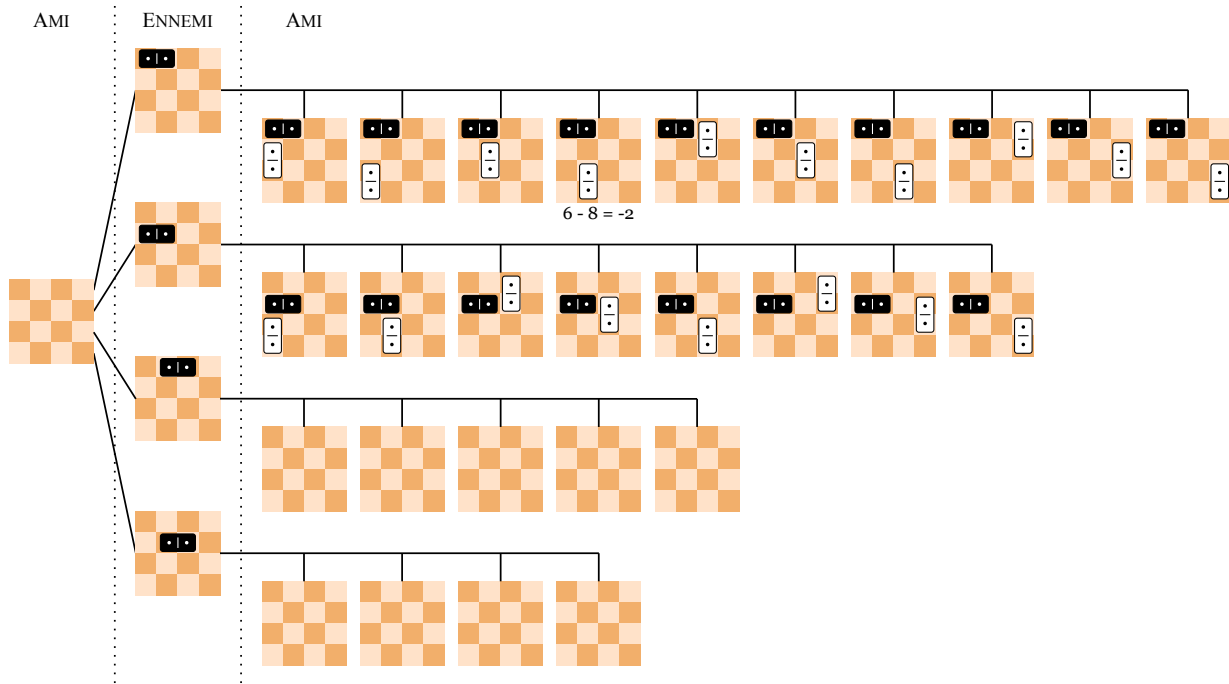
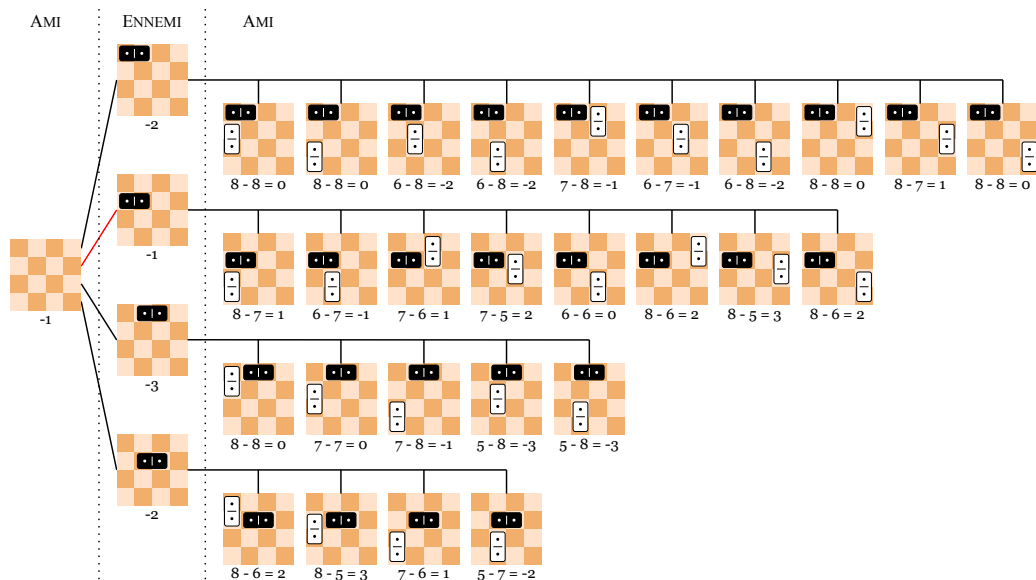


FIGURE 2 – Arbre de jeu à profondeur 2.



D'après l'algorithme MinMax à profondeur 2, le joueur horizontal devrait jouer :



2. Dans le calcul de f_H , pourquoi a-t-on besoin d'appeler $n.tour()$ dans le cas $(nbPlacementsHor = 0) \wedge (nbPlacementsVer = 0)$? Donnez un exemple de plateau de jeu correspondant à cette situation.

Correction : S'il n'y a plus aucun placement possible, la partie est terminée et le joueur qui s'apprêtait à jouer a perdu, puisque le dernier domino aura été posé par son adversaire. Dans ce cas, il faut donc déterminer qui a le trait (grâce à $n.tour()$) pour savoir si le joueur horizontal est celui qui a gagné ($f_H(n) = +\infty$) ou perdu ($f_H(n) = -\infty$).



3. Comment calculer f_V , la fonction d'évaluation selon le point de vue du joueur vertical? Écrire un algorithme complet ne sera pas nécessaire.

Correction : De façon assez simple, $f_V = -f_H$; cela revient aussi à intervertir les rôles (horizontal et vertical) dans la fonction d'évaluation.

Annexe

Algorithme 2 Fonction d'évaluation pour le joueur horizontal

Entrée : n : un nœud de l'arbre de jeu, pour lequel on dispose des méthodes
 $\text{nbCoupsHor}()$, qui donne le nombre de coups possibles pour le joueur horizontal, c.-à-d. le nombre d'options pour placer un domino horizontalement,
 $\text{nbCoupsVer}()$, qui donne le nombre de coups possibles pour le joueur vertical, c.-à-d. le nombre d'options pour placer un domino verticalement
 $\text{tour}()$, qui renvoie le joueur dont c'est le tour (JOUERHORIZONTAL ou JOUEURVERTICAL)

Sortie : valeur dans $\mathbb{R}$ correspondant à la qualité de n du point de vue du joueur horizontal

fonction $f_H(n)$
 $\text{nbPlacementsHor} \leftarrow n.\text{nbCoupsHor}()$
 $\text{nbPlacementsVer} \leftarrow n.\text{nbCoupsVer}()$
si $\text{nbPlacementsHor} = 0$ **et** $\text{nbPlacementsVer} = 0$ **alors**
 si $n.\text{tour}() = \text{JOUERHORIZONTAL}$ **alors**
 retourner $-\infty$
 sinon /* c.-à-d. $n.\text{tour}() = \text{JOUERVERTICAL}$ */
 retourner $+\infty$
 fin si
sinon si $\text{nbPlacementsHor} = 0$ **alors**
 retourner $-\infty$
sinon si $\text{nbPlacementsVer} = 0$ **alors**
 retourner $+\infty$
sinon
 retourner $\text{nbPlacementsHor} - \text{nbPlacementsVer}$
fin si
fin fonction

Exercice 3 Partie abstraite

Considérons l'arbre de jeu représenté par la figure 3. Par convention, la racine est un nœud AMI (ou MAX). Chaque nœud ou feuille est identifié par une lettre, et à chaque feuille est associée une évaluation (correspondant au point de vue AMI).

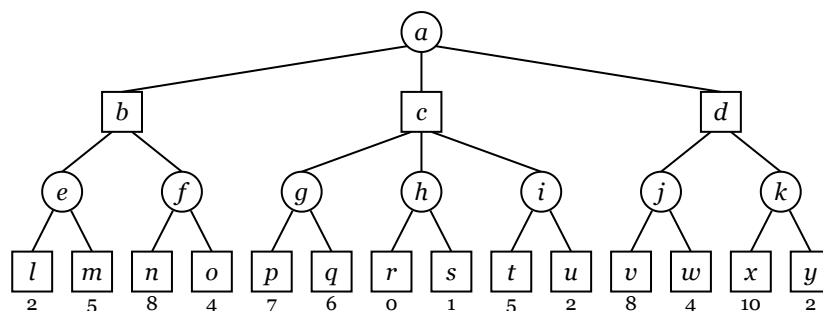
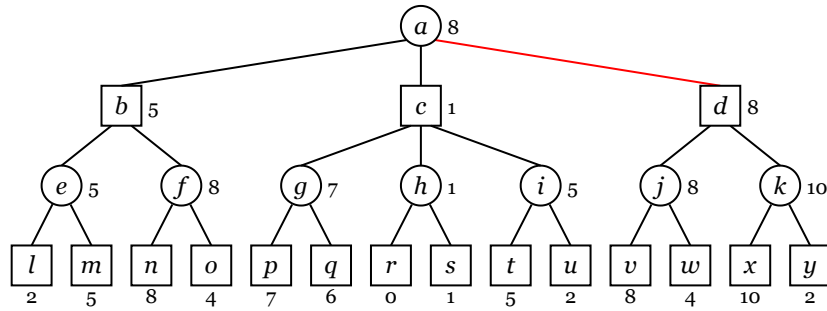


FIGURE 3 – Arbre de jeu abstrait.

1. Appliquez rapidement l'algorithme MinMax (ne recopiez pas l'arbre, utilisez directement celui de la feuille d'énoncé). Quel est le meilleur coup ?

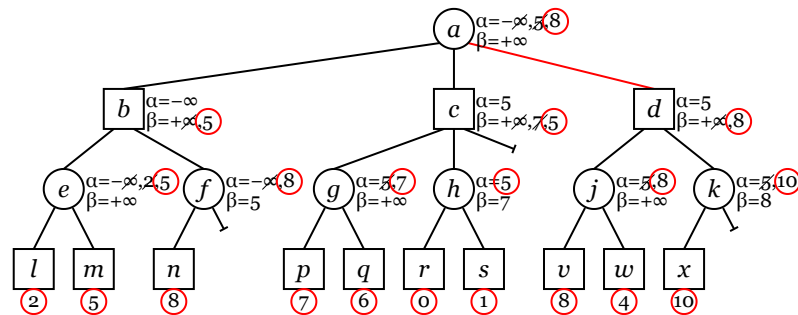
Correction :



Le meilleur coup serait de jouer d .

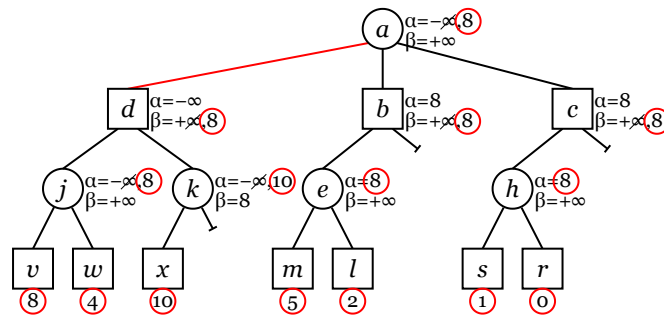
- Appliquez l'algorithme AlphaBeta, en supposant (comme on le fait habituellement) que les fils d'un nœud sont à parcourir de gauche à droite.

Correction :



- Appliquez l'algorithme AlphaBeta, en supposant que les fils d'un nœud AMI sont à parcourir par ordre de valeurs décroissantes, et que les fils d'un nœud ENNEMI sont à parcourir par ordre de valeurs croissantes (on se référera aux valeurs trouvées lors de la première question).

Correction :



- Commentez.

Correction : L'arrangement judicieux des fils permet de déclencher davantage de coupes.

Annexe

Algorithme 3 MinMax

Entrée :

- n : un nœud de l'arbre de jeu
- p : profondeur du nœud n
- $p_{\max}$: profondeur d'exploration maximale
- f : fonction d'évaluation heuristique (point de vue AMI)

Sortie : évaluation du nœud n à horizon $p_{\max} - p$

```
fonction minMax( $n, p, p_{\max}, f$ )  
  si estFinal( $n$ ) ou  $p = p_{\max}$  alors  
    retourner  $f(n)$   
  sinon si tour( $n$ ) = AMI alors /* tour AMI (max) */  
     $v_{\max} \leftarrow -\infty$   
    pour tout  $n' \in \text{successeurs}(n)$  faire  
       $v \leftarrow \text{minMax}(n', p + 1, p_{\max}, f)$   
       $v_{\max} \leftarrow \max(v_{\max}, v)$   
    fin pour  
    retourner  $v_{\max}$   
  sinon /* tour ENNEMI (min) */  
     $v_{\min} \leftarrow +\infty$   
    pour tout  $n' \in \text{successeurs}(n)$  faire  
       $v \leftarrow \text{minMax}(n', \alpha, \beta, p + 1, p_{\max}, f)$   
       $v_{\min} \leftarrow \min(v_{\min}, v)$   
    fin pour  
    retourner  $v_{\min}$   
fin si  
fin fonction
```

Algorithme 4 Initialisation MinMax

Entrée :

- n_{init} : racine de l'arbre de jeu
- $p_{\max}$: profondeur d'exploration maximale
- f : fonction d'évaluation heuristique (point de vue AMI)

Sortie : meilleur coup à jouer (selon évaluation à horizon $p_{\max}$)

```
fonction minMaxInit( $n_{\text{init}}, p_{\max}, f$ )  
   $\text{coups} \leftarrow \text{coupsPossibles}(n_{\text{init}})$   
   $\text{meilleurCoup} \leftarrow \text{coups}[0]$   
  /* 1er tour AMI */  
   $v_{\max} \leftarrow -\infty$   
  pour tout  $c \in \text{coups}$  faire  
     $n' \leftarrow \text{jouer}(n_{\text{init}}, c)$  /*  $n'$  successeur de  $n_{\text{init}}$ , via  $c$  */  
     $v \leftarrow \text{minMax}(n', 1, p_{\max}, f)$   
    si  $v > v_{\max}$  alors  
       $v_{\max} \leftarrow v$   
       $\text{meilleurCoup} \leftarrow c$   
    fin si  
  fin pour  
  retourner  $\text{meilleurCoup}$   
fin fonction
```

Algorithme 5 AlphaBeta

Entrée :

- n : un nœud de l'arbre de jeu
- α, β : bornes du score que le joueur AMI peut espérer
- p : profondeur du nœud n
- $p_{\max}$: profondeur d'exploration maximale
- f : fonction d'évaluation heuristique (point de vue AMI)

Sortie : évaluation du nœud n à horizon $p_{\max} - p$

```
fonction alphaBeta( $n, \alpha, \beta, p, p_{\max}, f$ )  
  si estFinal( $n$ ) ou  $p = p_{\max}$  alors  
    retourner  $f(n)$   
  sinon si tour( $n$ ) = AMI alors /* tour AMI (max) : m-à-j de  $\alpha$  */  
    pour tout  $n' \in \text{successeurs}(n)$  faire  
       $v \leftarrow \text{alphaBeta}(n', \alpha, \beta, p + 1, p_{\max}, f)$   
       $\alpha \leftarrow \max(\alpha, v)$   
      si  $\alpha \geq \beta$  alors rompre /* coupe */  
    fin si  
  fin pour  
  retourner  $\alpha$   
  sinon /* tour ENNEMI (min) : m-à-j de  $\beta$  */  
    pour tout  $n' \in \text{successeurs}(n)$  faire  
       $v \leftarrow \text{alphaBeta}(n', \alpha, \beta, p + 1, p_{\max}, f)$   
       $\beta \leftarrow \min(\beta, v)$   
      si  $\alpha \geq \beta$  alors rompre /* coupe */  
    fin si  
  fin pour  
  retourner  $\beta$   
fin si  
fin fonction
```

Algorithme 6 Initialisation AlphaBeta

Entrée :

- n_{init} : racine de l'arbre de jeu
- $p_{\max}$: profondeur d'exploration maximale
- f : fonction d'évaluation heuristique (point de vue AMI)

Sortie : meilleur coup à jouer (selon évaluation à horizon $p_{\max}$)

```
fonction alphaBetaInit( $n_{\text{init}}, p_{\max}, f$ )  
   $\alpha \leftarrow -\infty$   
   $\beta \leftarrow +\infty$   
   $\text{coups} \leftarrow \text{coupsPossibles}(n_{\text{init}})$   
   $\text{meilleurCoup} \leftarrow \text{coups}[0]$   
  /* 1er tour AMI */  
  pour tout  $c \in \text{coups}$  faire  
     $n' \leftarrow \text{jouer}(n_{\text{init}}, c)$  /*  $n'$  successeur de  $n_{\text{init}}$ , via  $c$  */  
     $v \leftarrow \text{alphaBeta}(n', \alpha, \beta, 1, p_{\max}, f)$   
    si  $v > \alpha$  alors  
       $\alpha \leftarrow v$   
       $\text{meilleurCoup} \leftarrow c$   
    fin si  
  fin pour  
  retourner  $\text{meilleurCoup}$   
fin fonction
```
